

+ TEMA 11

MINERÍA DE DATOS

EN PYTHON

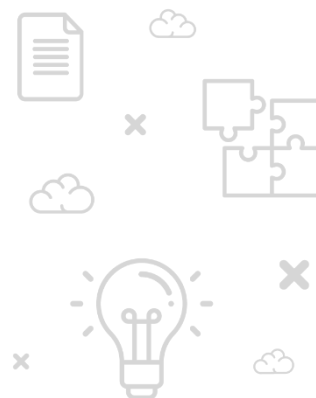
- Algoritmos: clasificación, clustering con scikit-learn
- Visualización: gráficos con matplotlib, seaborn
- Evaluación de modelos: métricas, validación cruzada
- Preprocesamiento: pandas para limpieza de datos
- Optimización: ajuste de hiperparámetros





+ INTRODUCCIÓN **DE LA SESIÓN**

- + La minería de datos en Python se ha convertido en una herramienta fundamental para el análisis avanzado de información en contextos empresariales y científicos.
- + Python destaca por su simplicidad, flexibilidad y por contar con una amplia variedad de bibliotecas especializadas en análisis de datos y aprendizaje automático.
- + Bibliotecas como pandas, scikit-learn, matplotlib y seaborn han estandarizado los procesos de análisis, modelado y visualización de datos. Gracias a ellas, Python es ampliamente utilizado en entornos profesionales, ayudando a la minería de datos a descubrir patrones, tendencias y relaciones ocultas en grandes volúmenes de información.
- + Estos hallazgos facilitan la toma de decisiones estratégicas basadas en evidencia, lo que reduce la incertidumbre y mejora la competitividad organizacional.



+

ALGORITMOS:

**CLASIFICACIÓN, CLUSTERING
CON SCIKIT-LEARN**

SCIKIT-LEARN (SKLEARN)

- Es una biblioteca de machine learning de código abierto para Python, diseñada para implementar de manera eficiente los principales algoritmos de minería de datos y aprendizaje automático.
- Se caracteriza por lo siguiente:
 - Su facilidad de uso.
 - Consistencia en la sintaxis.
 - Su integración con otras bibliotecas como pandas, NumPy y matplotlib.



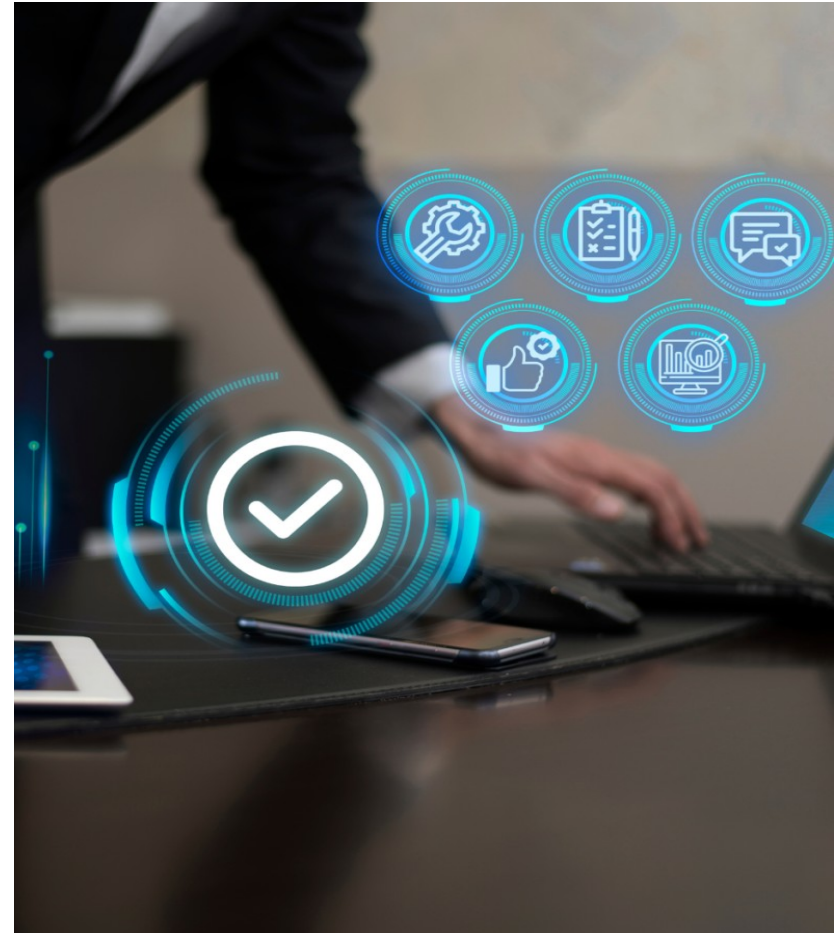
SCIKIT-LEARN (SKLEARN)

- En minería de datos, **scikit-learn** permite aplicar algoritmos de **clasificación y clustering** siguiendo un flujo de trabajo estandarizado.
- Esto facilita la comprensión del proceso analítico y su aplicación en contextos profesionales reales.



CLASIFICACIÓN

- Es una técnica de aprendizaje supervisado que consiste en predecir la categoría o clase a la que pertenece una observación, a partir de datos históricos que incluyen una variable objetivo conocida.
- Por ejemplo:
 - **Cliente: compra / no compra.**
 - **Crédito: aprobado / rechazado.**
 - **Riesgo financiero: bajo / medio / alto.**
- En scikit-learn, la clasificación se implementa mediante modelos que aprenden patrones entre las variables predictoras y las clases.



ALGORITMOS DE CLASIFICACIÓN MÁS USADOS EN SCIKIT-LEARN

REGRESIÓN LOGÍSTICA

- Modelo estadístico supervisado que estima la probabilidad de pertenecer a una clase.
- Por ejemplo: predecir si un cliente comprará un producto según su edad e ingresos.

ÁRBOLES DE DECISIÓN

- Algoritmo que clasifica mediante reglas tipo “si-entonces” organizadas en forma de árbol.
- Por ejemplo: aprobar o rechazar un crédito según ingresos y nivel de endeudamiento.

ALGORITMOS DE CLASIFICACIÓN MÁS USADOS EN SCIKIT-LEARN

K-NEAREST NEIGHBORS (K-NN)

- Clasifica una observación según la clase mayoritaria de sus k vecinos más cercanos.
- Por ejemplo: clasificar un cliente según el comportamiento de clientes similares.

SUPPORT VECTOR MACHINES (SVM)

- Algoritmo que separa clases encontrando el hiperplano óptimo que maximiza la distancia entre ellas.
- Por ejemplo: detectar correos fraudulentos a partir de características del mensaje.

EJEMPLO APLICADO DE CLASIFICACIÓN EN MARKETING

- Una empresa de marketing digital desea optimizar sus campañas publicitarias identificando qué clientes tienen mayor probabilidad de comprar un nuevo producto. Para ello, cuenta con datos históricos de clientes que incluyen su edad, nivel de ingresos mensuales y si realizaron o no la compra.
- El problema consiste en construir un modelo de clasificación que permita predecir si un nuevo cliente comprará el producto, de modo que la empresa pueda enfocar sus recursos publicitarios en los clientes con mayor probabilidad de conversión.
- La variable objetivo es:
 - **Compra: 1 = compra el producto, 0 = no compra.**



DESARROLLO DEL MODELO DE CLASIFICACIÓN CON SCIKIT-LEARN

Paso 1. Definición del problema

- Tipo de problema: clasificación supervisada.
- Objetivo: predecir si un cliente comprará un producto.
- Algoritmo elegido: regresión logística (por su simplicidad e interpretación).

Paso 2. Carga y exploración de los datos en Python

```
import pandas as pd
# Crear el conjunto de datos
data = {
    'edad': [22, 25, 47, 52, 46, 56, 30, 35],
    'ingresos': [1200, 1500, 3500, 4000, 3800, 4200, 2000, 2500],
    'compra': [0, 0, 1, 1, 1, 1, 0, 0]
}

df = pd.DataFrame(data)
df
```

Paso 3. Separación de variables predictoras y variable objetivo

Tener en cuenta que:

- "X" contiene las características del cliente.
- "Y" indica si el cliente compró o no el producto.

```
X = df[['edad', 'ingresos']] # Variables explicativas
y = df['compra']           # Variable objetivo
```

Paso 4. División del conjunto de datos (entrenamiento y prueba)

Recomendación: el 75% de los datos se utiliza para entrenar el modelo y el 25% para evaluar su desempeño con datos no vistos.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

Paso 5. Entrenamiento del modelo de clasificación

La regresión logística aprende la relación entre edad, ingresos y la probabilidad de compra del cliente.

```
from sklearn.linear_model import LogisticRegression

modelo = LogisticRegression()
modelo.fit(X_train, y_train)
```

Paso 6. Predicción con el modelo

El modelo clasifica a los clientes del conjunto de prueba como compradores (1) o no compradores (0).

```
y_pred = modelo.predict(X_test)
y_pred
```

Paso 7. Evaluación del modelo

El resultado esperado es 0.80, entonces se puede decir que el modelo clasifica correctamente el 80% de los clientes, lo que indica un buen desempeño inicial para apoyar decisiones de marketing.

```
from sklearn.metrics import accuracy_score  
  
accuracy_score(y_test, y_pred)
```

Paso 8. Predicción de un nuevo cliente

```
nuevo_cliente = pd.DataFrame({  
    'edad': [40],  
    'ingresos': [3000]  
})  
  
modelo.predict(nuevo_cliente)  
modelo.predict_proba(nuevo_cliente)
```

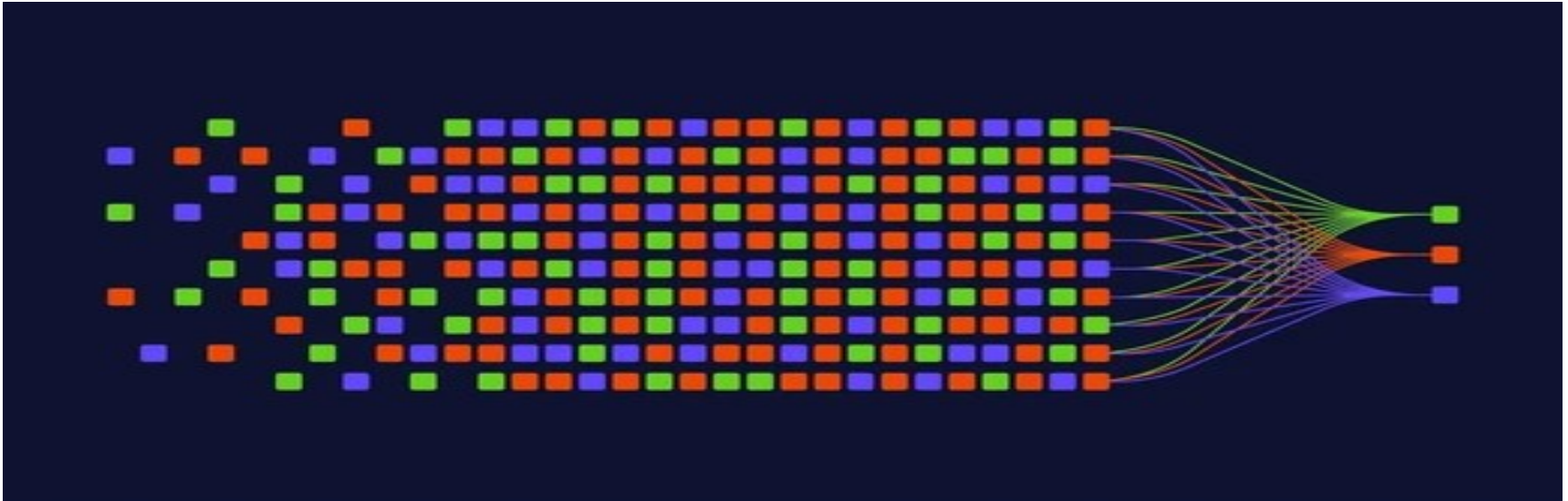
RESULTADO

Predicción: 1
Probabilidad: [0.22, 0.78]

- El modelo predice que un cliente de 40 años con ingresos de 3,000 tiene una probabilidad del 78% de comprar el producto. Desde el punto de vista empresarial, este cliente debería ser priorizado en campañas promocionales, ya que presenta una alta probabilidad de conversión.
- Este tipo de modelo permite a las empresas lo siguiente:
 - **Optimizar el presupuesto publicitario.**
 - **Mejorar la segmentación de clientes.**
 - **Aumentar la efectividad de las estrategias de marketing basadas en datos.**

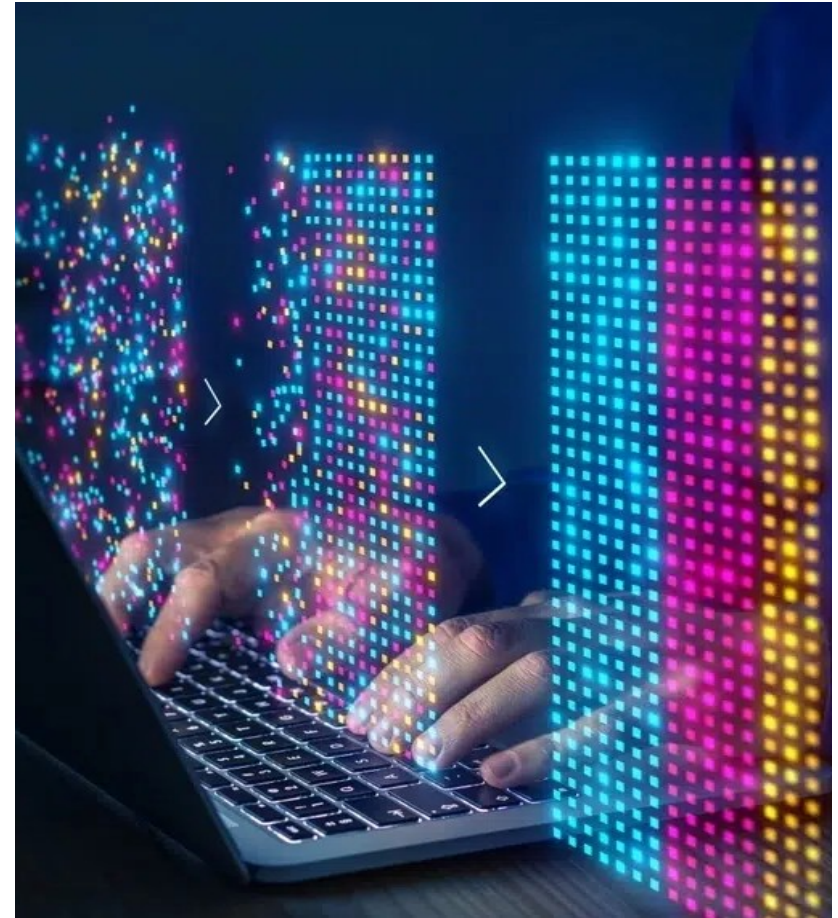
CLUSTERING CON SCIKIT-LEARN

- El clustering es una técnica de aprendizaje no supervisado **cuyo objetivo es agrupar observaciones similares sin necesidad de una variable objetivo previamente definida.**
- En scikit-learn, el clustering permite descubrir estructuras ocultas en los datos mediante algoritmos que miden la similitud entre observaciones.



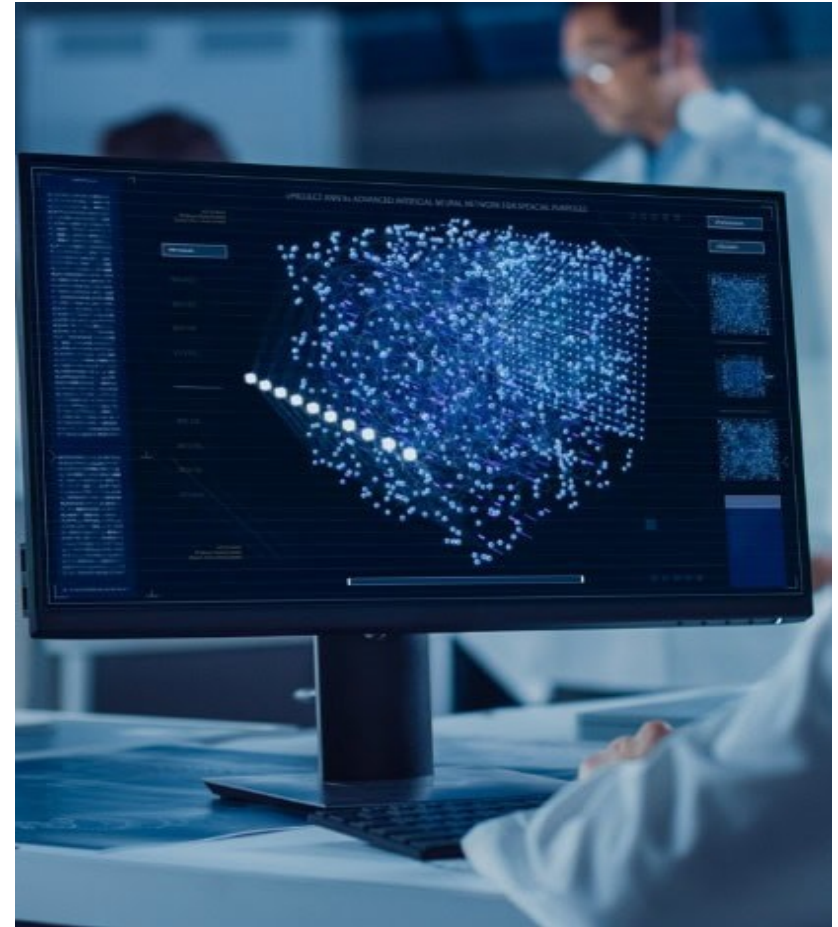
LIBRERÍA SCIKIT-LEARN

- Proporciona una implementación estandarizada y eficiente de los principales algoritmos de clustering. En ese sentido, **k-means es el más utilizado en contextos empresariales** por su simplicidad, rapidez e Interpretabilidad.
- Desde una perspectiva profesional, el clustering no busca predecir, sino comprender y segmentar, lo que lo convierte en una herramienta clave para el análisis exploratorio de datos.



CARACTERÍSTICAS PRINCIPALES DEL CLUSTERING EN SCIKIT-LEARN

- No requiere etiquetas ni variable dependiente.
- Trabaja a partir de distancias o similitudes entre observaciones.
- Requiere preprocesamiento, especialmente escalamiento de variables.
- Los resultados deben ser interpretados por el analista, no son automáticos.
- Permite integrar fácilmente visualización y evaluación.



EJEMPLO APLICADO DE CLUSTERING CON SCIKIT-LEARN (PYTHON)

- Una empresa de comercio electrónico desea segmentar a sus clientes para mejorar sus estrategias comerciales. Para ello, se cuenta con información de:
 - **Gasto mensual promedio (USD).**
 - **Frecuencia de compra mensual.**
- El objetivo es identificar grupos de clientes con comportamientos similares para aplicar estrategias diferenciadas.



DESARROLLO PASO A PASO DEL CLUSTERING EN SCIKIT-LEARN

Paso 1. Importar librerías necesarias

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

Paso 2. Crear el conjunto de datos

```
data = {
    'gasto_mensual': [100, 120, 150, 200, 220, 250, 400, 420, 450, 480],
    'frecuencia_compra': [1, 1, 2, 2, 3, 3, 5, 5, 6, 6]
}

df = pd.DataFrame(data)
df
```

Paso 3. Estandarización de los datos

En scikit-learn, k-means es sensible a la escala, por lo que es obligatorio estandarizar los datos.

```
scaler = StandardScaler()  
df_scaled = scaler.fit_transform(df)
```

Paso 4. Definir el número de clusters (k)

Para este ejemplo, se asume que la empresa desea identificar 3 segmentos de clientes.

```
) kmeans = KMeans(n_clusters=3, random_state=42)
```

Paso 5. Entrenamiento del modelo

```
kmeans.fit(df_scaled)
```

Paso 6. Asignación de clusters a los clientes

```
df['cluster'] = kmeans.labels_  
df
```

Paso 7. Visualización de los clusters

```
plt.scatter(df['gasto_mensual'], df['frecuencia_compra'], c=df['cluster'])  
plt.xlabel('Gasto mensual')  
plt.ylabel('Frecuencia de compra')  
plt.title('Segmentación de clientes')  
plt.show()
```

Resultado

El modelo genera tres grupos claramente diferenciados:

1. Clientes de bajo gasto y baja frecuencia.
2. Clientes de gasto y frecuencia media.
3. Clientes de alto gasto y alta frecuencia.

Cada cliente queda asignado a un cluster específico.

INTERPRETACIÓN DEL CLUSTERING Y RECOMENDACIONES

CLUSTER 0 - CLIENTES DE BAJO VALOR

1

- Bajo gasto y compras esporádicas.
- Estrategia: promociones, descuentos y campañas de activación.

CLUSTER 1 - CLIENTES DE VALOR MEDIO

2

- Gasto moderado y frecuencia estable.
- Estrategia: programas de fidelización y ventas cruzadas.

CLUSTER 2 - CLIENTES DE ALTO VALOR

3

- Alto gasto y elevada frecuencia de compra.
- Estrategia: beneficios exclusivos, atención personalizada y retención.

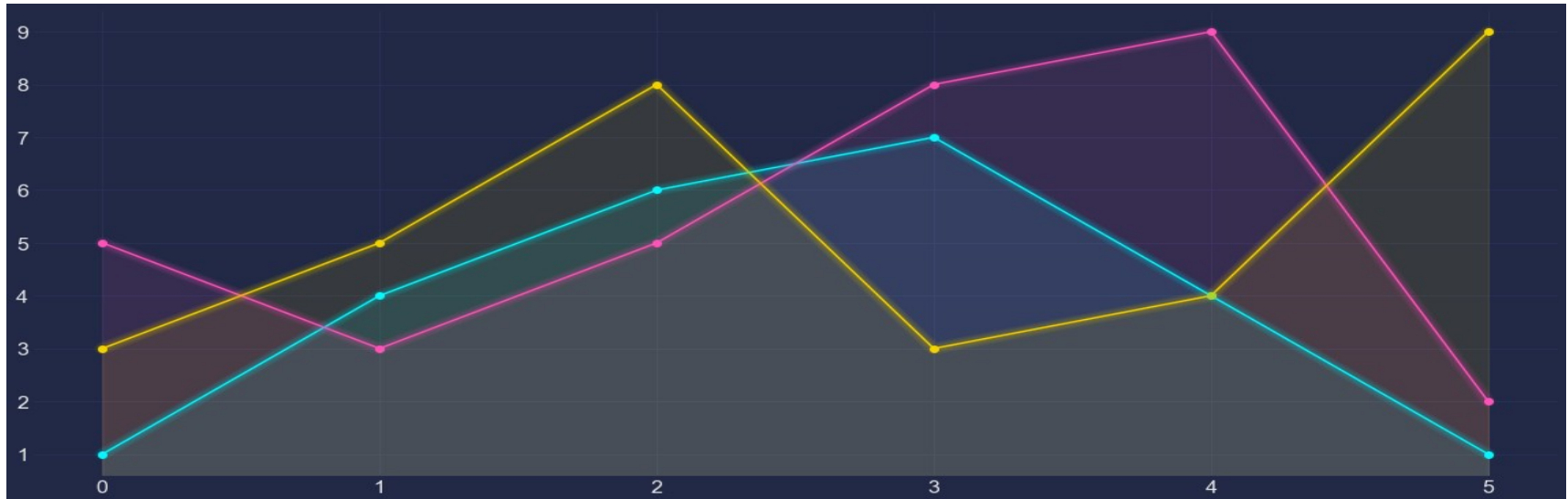
+

VISUALIZACIÓN:

**GRÁFICOS CON MATPLOTLIB,
SEABORN**

VISUALIZACIÓN DE DATOS

- Es el proceso de representar información numérica mediante gráficos, con el objetivo de facilitar la comprensión, el análisis y la comunicación de resultados.
- En análisis estadístico y minería de datos, **la visualización es una etapa clave porque permite identificar patrones, tendencias, relaciones y valores atípicos** que no siempre son evidentes en tablas de datos



LIBRERÍAS DE VISUALIZACIÓN EN PYTHON

- En Python, las dos librerías más utilizadas para este fin son **matplotlib** y **seaborn**, ambas ampliamente empleadas en entornos académicos y profesionales.
- Estas herramientas permiten transformar datos complejos en representaciones visuales claras y comprensibles para la toma de decisiones.



+ 01. MATPLOTLIB

Es la librería base de visualización en Python. Permite crear gráficos desde cero y ofrece control total sobre cada elemento del gráfico.

Características:

- Muy flexible y personalizable.
- Ideal para gráficos básicos: líneas, barras, dispersión.
- Requiere más código para gráficos complejos.
- Es la base sobre la que se construye seaborn.



+ 02. SEABORN

Es una librería de visualización estadística construida sobre matplotlib. Está diseñada para crear gráficos más informativos y estéticos, con menos código.

Características:

- Integración directa con pandas.
- Gráficos estadísticos avanzados.
- Estilos visuales automáticos.
- Ideal para análisis exploratorio de datos.



PROCESO DE DESARROLLO DE UNA VISUALIZACIÓN EN PYTHON

- 01  Definir el objetivo del gráfico (comparar, analizar tendencias, identificar relaciones).
- 02  Seleccionar las variables adecuadas.
- 03  Elegir el tipo de gráfico.
- 04  Construir el gráfico con la librería adecuada.
- 05  Interpretar los resultados en contexto profesional.



EJEMPLO 1: VISUALIZACIÓN CON MATPLOTLIB

- Una empresa comercial desea evaluar el desempeño de sus ventas mensuales durante el primer semestre del año. El objetivo es identificar tendencias de crecimiento o disminución que permitan a la gerencia tomar decisiones sobre continuidad de campañas, ajustes de precios o planificación de inventarios.
- La gerencia necesita un gráfico claro y sencillo, fácil de interpretar y adecuado para un informe ejecutivo.
- Se dispone del total de ventas mensuales (en soles) registradas durante seis meses:

MES	VENTAS
Enero	12,000
Febrero	15,000
Marzo	18,000
Abril	17,000
Mayo	20,000
Junio	23,000

+ CÓDIGO PYTHON

```
import matplotlib.pyplot as plt

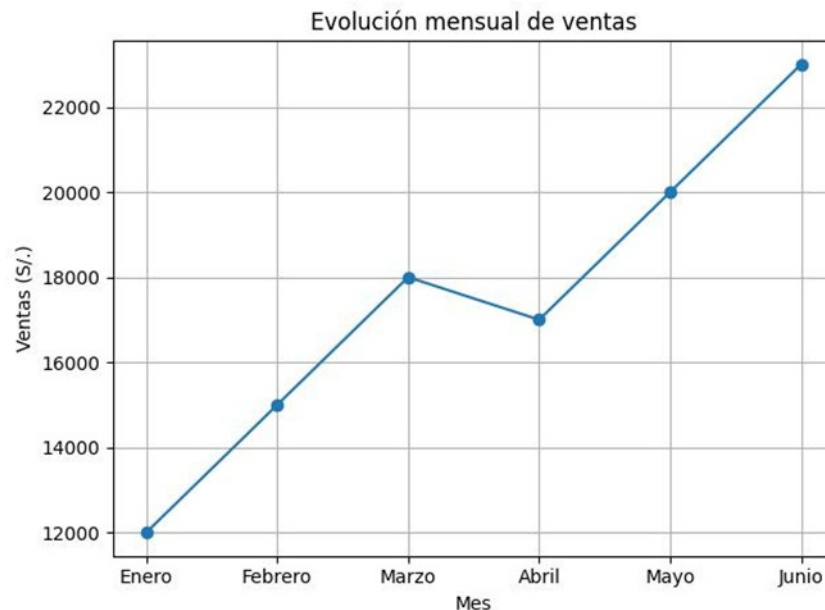
# Datos
meses = ['Enero', 'Febrero', 'Marzo', 'Abril', 'Mayo', 'Junio']
ventas = [12000, 15000, 18000, 17000, 20000, 23000]

# Gráfico
plt.plot(meses, ventas, marker='o')
plt.xlabel('Mes')
plt.ylabel('Ventas ($/.)')
plt.title('Evolución mensual de ventas')
plt.grid(True)

# Mostrar gráfico
plt.show()
```

GRÁFICA

- El gráfico evidencia una tendencia general de crecimiento en las ventas, lo cual indica un buen desempeño comercial durante el semestre.
- La leve disminución en abril puede estar asociada a factores estacionales o ajustes operativos, pero no altera la tendencia positiva global.
- Desde una perspectiva administrativa, los resultados respaldan la continuidad de la estrategia comercial, con especial énfasis en replicar las acciones implementadas a partir de mayo.



EJEMPLO 2: VISUALIZACIÓN CON SEABORN

- Una entidad financiera desea analizar la distribución de los montos de crédito otorgados a sus clientes. El objetivo es comprender qué rangos de crédito son más frecuentes, lo cual permitirá evaluar políticas de riesgo y segmentar clientes según su perfil financiero. Este análisis se realiza como parte de un estudio exploratorio de datos.
- Se cuenta con una muestra de montos de crédito otorgados (en soles):

CLIENTE	MONTO DE CRÉDITO
1	5,000
2	7,000
3	8,000
4	10,000
5	12,000
6	15,000
7	20,000

+ CÓDIGO PYTHON

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Datos
datos_credito = {
    'monto_credito': [5000, 7000, 8000, 10000, 12000, 15000, 20000]
}

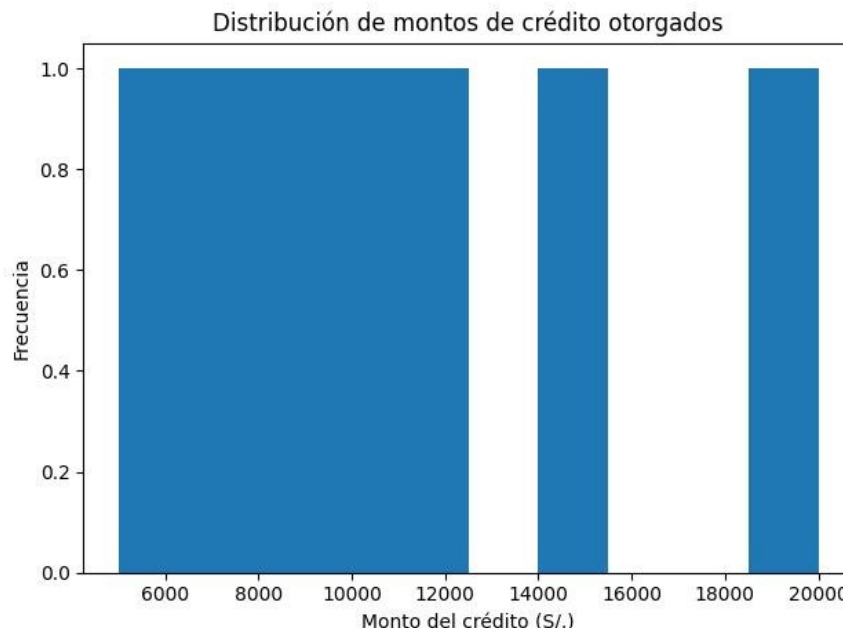
df = pd.DataFrame(datos_credito)

# Gráfico de distribución
sns.histplot(df['monto_credito'], kde=True)
plt.xlabel('Monto del crédito (S/.)')
plt.ylabel('Frecuencia')
plt.title('Distribución de montos de crédito otorgados')

# Mostrar gráfico
plt.show()
```

GRÁFICA

- El gráfico muestra que la mayoría de los créditos otorgados se concentran en rangos medios, lo que sugiere una política conservadora de otorgamiento.
- Los créditos de mayor monto aparecen con menor frecuencia, lo cual es coherente con estrategias de control de riesgo financiero.
- Esta información permite a la entidad financiera segmentar clientes, ajustar condiciones crediticias y diseñar productos específicos según el perfil observado.



+

EVALUACIÓN DE
**MODELOS: MÉTRICAS,
VALIDACIÓN CRUZADA**

EVALUACIÓN DE MODELOS

- Es el **proceso en el que se mide qué tan bien un modelo de minería de datos realiza predicciones sobre datos nuevos**. El objetivo principal es determinar si el modelo es confiable, preciso y útil.
- Esto implica observar si “funciona”, cuantificar su rendimiento, compararlo con otros modelos y verificar que no esté aprendiendo de memoria los datos.
- En Python, la evaluación se realiza principalmente con la librería scikit-learn, que proporciona métricas estandarizadas y métodos automáticos para validar modelos predictivos.



MÉTRICAS DE EVALUACIÓN

- Las métricas son indicadores numéricos que permiten medir el desempeño del modelo.
- La métrica adecuada depende del tipo de problema (clasificación, regresión o clustering) y del contexto profesional.

Importancia de las métricas

- + **Permiten comparar modelos de forma objetiva.**
- + **Ayudan a seleccionar el mejor modelo para un problema real.**
- + **Evitan decisiones empresariales basadas en modelos poco confiables.**
- + **Traducen resultados técnicos en indicadores comprensibles para la gestión.**



MÉTRICAS PARA CLASIFICACIÓN

En problemas de clasificación, el modelo predice una categoría (por ejemplo: cliente compra / no compra).

MÉTRICA	DESCRIPCIÓN
Accuracy (exactitud)	Proporción total de predicciones correctas.
Precisión	Qué tan confiables son las predicciones positivas.
Recall (sensibilidad)	Qué tanto del total real fue correctamente identificado.
F1-score	Equilibrio entre precisión y recall.

EJEMPLO 1: PREDICCIÓN DE COMPRA

- Una empresa de marketing digital busca mejorar la efectividad de sus campañas comerciales. Para ello, desea anticipar el comportamiento de los usuarios que visitan su sitio web, específicamente predecir si un cliente realizará una compra o no.
- La empresa dispone de datos históricos simples sobre la interacción de los usuarios con la página web, tales como el número de visitas realizadas y el tiempo total de permanencia en el sitio.
- Estas variables reflejan el nivel de interés del cliente y son comúnmente utilizadas en análisis de comportamiento digital.

Cliente	visitas_web	tiempo_web	compra
1	1	2	0
2	3	4	0
3	5	6	1
4	2	3	0
5	8	9	1
6	7	8	1
7	4	5	0
8	6	7	1

EJEMPLO 1: PREDICCIÓN DE COMPRA

- El objetivo es construir en Python un modelo predictivo que, a partir del comportamiento previo del usuario, permita clasificar nuevos clientes y apoyar la toma de decisiones en campañas de marketing. Para el desarrollo del análisis en Python, se utilizará un conjunto de datos estructurado, compuesto por las siguientes variables:
 - **visitas_web**: número de veces que el cliente ingresó al sitio web.
 - **tiempo_web**: tiempo total (en horas) que el cliente permaneció navegando en el sitio.
 - **compra**: variable objetivo que indica si el cliente realizó una compra (1) o no (0).
- Estos datos serán procesados con la librería pandas, el modelo de clasificación se implementará con scikit-learn y su desempeño será evaluado mediante métricas de clasificación, con el fin de determinar qué tan confiable es el modelo para su uso en un entorno empresarial real.

+ DESARROLLO DEL CÓDIGO EN PYTHON

Paso 1. Importar librerías

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

Paso 2. Crear un dataset simple

```
data = {
    'visitas_web': [1, 3, 5, 2, 8, 7, 4, 6],
    'tiempo_web': [2, 4, 6, 3, 9, 8, 5, 7],
    'compra': [0, 0, 1, 0, 1, 1, 0, 1]
}

df = pd.DataFrame(data)
```

Paso 3. Separar variables predictoras y variable objetivo

```
X = df[['visitas_web', 'tiempo_web']]  
y = df['compra']
```

Paso 4. División en datos de entrenamiento y prueba

Recomendación: el 75% de los datos se utiliza para entrenar el modelo y el 25% para evaluar su desempeño con datos no vistos.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42  
)
```

Paso 5. Entrenar el modelo

```
model = LogisticRegression()  
model.fit(X_train, y_train)
```

Paso 6. Predicciones y evaluación

```
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

print(classification_report(y_test, y_pred))
```

Resultado en Python

Accuracy: 0.75

	precision	recall	f1-score	support
0	0.67	1.00	0.80	2
1	1.00	0.50	0.67	2

El modelo tiene una exactitud del 75%, lo que indica un desempeño aceptable para un conjunto pequeño de datos. La precisión alta en clientes compradores indica que, cuando el modelo predice compra, suele acertar. El recall menor indica que algunos compradores reales no fueron detectados.

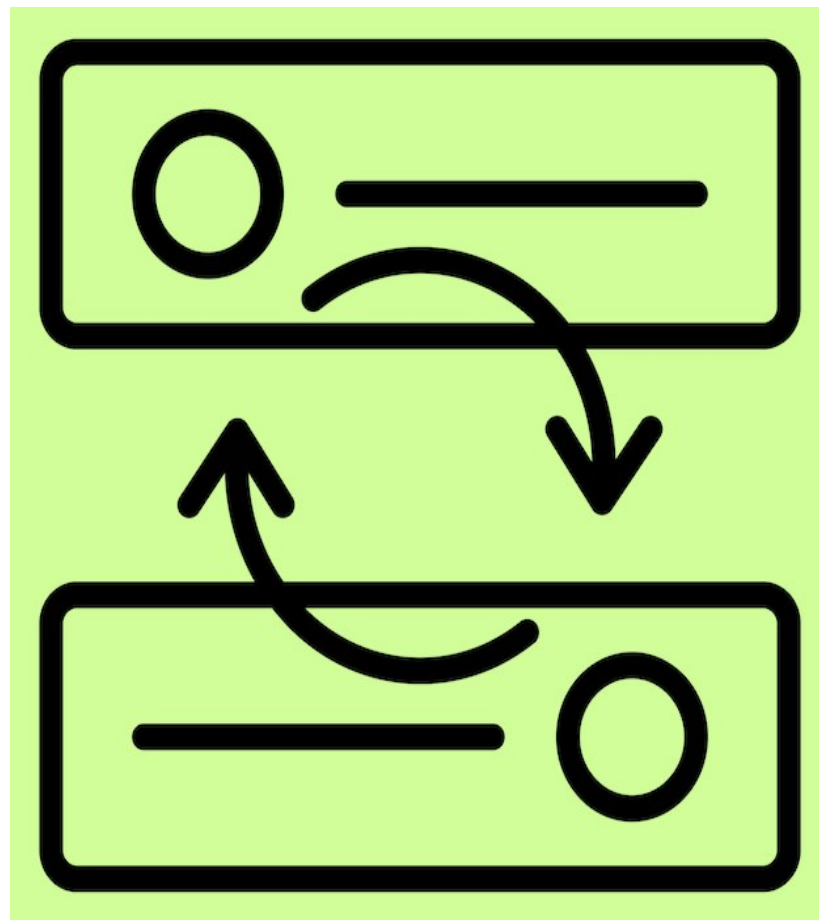
VALIDACIÓN CRUZADA

Es una técnica que evalúa el modelo varias veces usando diferentes particiones de los datos, en lugar de una sola división entrenamiento-prueba.

Características principales

- Reduce el riesgo de sobreajuste.
- Proporciona una evaluación más estable y confiable.
- Es ideal cuando se trabaja con pocos datos.
- Se integra fácilmente en Python con scikit-learn.

La técnica más común es k-fold cross-validation.



DIFERENCIAS CLAVE: MÉTRICAS VS. VALIDACIÓN CRUZADA

MÉTRICAS	VALIDACIÓN CRUZADA
Evalúan el rendimiento.	Evalúa la estabilidad del modelo.
Se calculan sobre un conjunto.	Usa múltiples subconjuntos.
Miden precisión, error, acierto.	Reduce sobreajuste.
Resultado puntual.	Resultado más confiable.

EJEMPLO 2: PREDICCIÓN DE COMPRA

Una entidad financiera busca reducir el riesgo de morosidad y mejorar la calidad de sus decisiones crediticias. Para ello, se ha desarrollado un modelo predictivo que permite estimar si un cliente cumplirá con el pago de su crédito (1) o incurrirá en incumplimiento (0), a partir de información histórica básica del solicitante.

Las variables son las siguientes:

- **ingreso_mensual:** ingreso promedio del cliente (en miles de unidades monetarias).
- **deuda_actual:** monto total de deudas vigentes del cliente.
- **pago_credito:** variable objetivo que indica si el cliente pagó el crédito (1) o no lo pagó (0).

Cliente	ingreso_mensual	deuda_actual	pago_credito
1	3.0	1.0	1
2	2.5	2.0	0
3	4.0	1.5	1
4	1.8	2.5	0
5	5.0	1.0	1
6	3.5	2.2	1
7	2.0	3.0	0
8	4.5	1.8	1

+ DESARROLLO DEL CÓDIGO EN PYTHON

Paso 1. Importar librerías

- **Pandas:** manejo de datos en forma de tablas.
- **LogisticRegression:** modelo de clasificación adecuado para problemas financieros binarios.
- **cross_val_score:** herramienta de validación cruzada.

```
import pandas as pd
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
```

Paso 2. Creación del dataset financiero

```
data = {
    'ingreso_mensual': [3.0, 2.5, 4.0, 1.8, 5.0, 3.5, 2.0, 4.5],
    'deuda_actual': [1.0, 2.0, 1.5, 2.5, 1.0, 2.2, 3.0, 1.8],
    'pago_credito': [1, 0, 1, 0, 1, 1, 0, 1]
}

df = pd.DataFrame(data)
print(df)
```

Paso 3. Definición de variables predictoras y variable objetivo

X: información financiera del cliente.

Y: comportamiento real de pago del crédito.

```
X = df[['ingreso_mensual', 'deuda_actual']]  
y = df['pago_credito']
```

Paso 4. Definición del modelo

La regresión logística es ampliamente usada en scoring crediticio, ya que estima probabilidades de cumplimiento.

```
model = LogisticRegression()
```

Paso 5. Aplicación de Validación Cruzada (k = 5)

```
scores = cross_val_score(  
    model,  
    X,  
    y,  
    cv=5,  
    scoring='accuracy'  
)  
  
print("Accuracy por cada fold:", scores)  
print("Accuracy promedio:", scores.mean())
```

Resultado en Python

```
Accuracy por cada fold: [0.75 0.75 1.00 0.50 0.75]  
Accuracy promedio: 0.75
```

El modelo fue evaluado 5 veces, usando diferentes combinaciones de datos. La exactitud promedio del 75% indica que el modelo clasifica correctamente 3 de cada 4 clientes. La variabilidad entre folds (0.50 a 1.00) muestra que el desempeño no depende de una sola partición, lo cual es positivo.

+

PREPROCESAMIENTO:
**PANDAS PARA
LIMPIEZA DE DATOS**

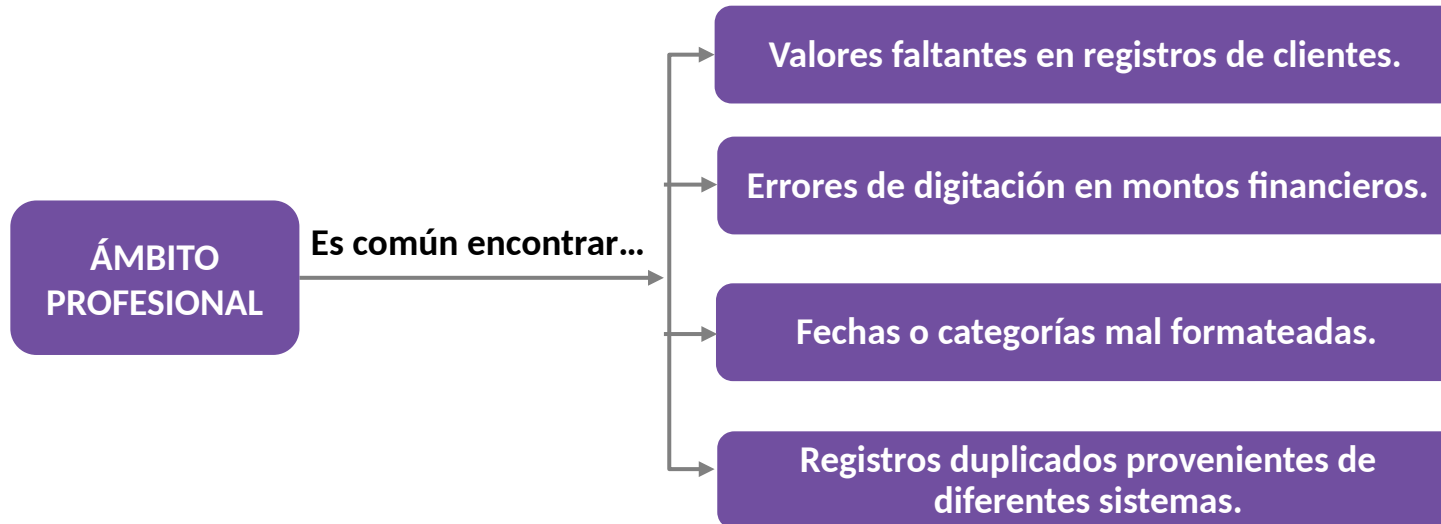
PREPROCESAMIENTO DE DATOS

- Es la etapa inicial y más crítica del proceso de minería de datos y análisis estadístico. **Consiste en revisar, limpiar y transformar los datos para que puedan ser analizados correctamente** por métodos estadísticos o algoritmos de aprendizaje automático.
- En Python, la biblioteca pandas es la herramienta principal para esta tarea, ya que permite trabajar con datos estructurados en forma de tablas (DataFrames), lo que facilita la detección de errores, valores faltantes, duplicados o formatos inconsistentes.



IMPORTANCIA DEL PREPROCESAMIENTO DE DATOS

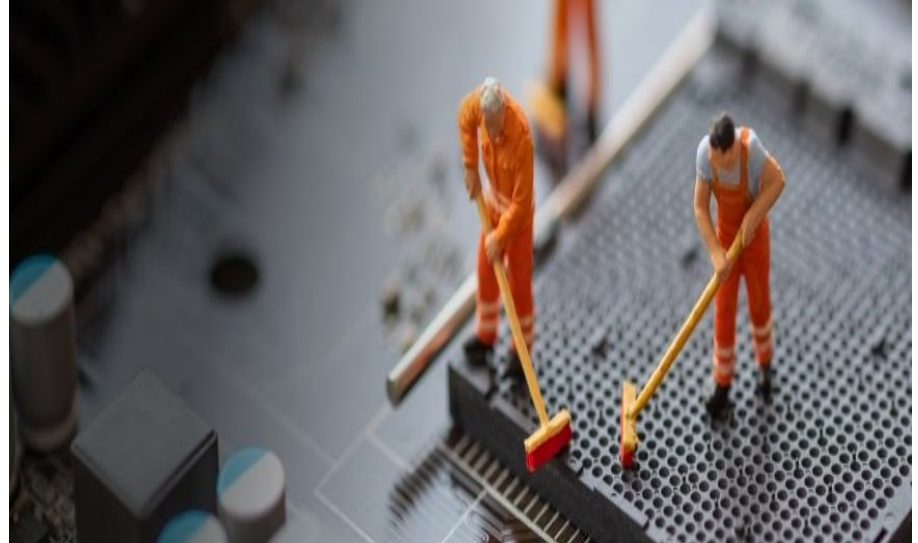
- Un conjunto de datos sin preprocesar puede generar resultados erróneos o poco confiables. Por ello, el preprocesamiento garantiza que los datos sean coherentes, completos y adecuados para el análisis posterior.
- En el ámbito profesional, los datos rara vez se presentan en condiciones ideales.



USO DE PANDAS

- Mejora la calidad de la información.
- Reduce riesgos en la toma de decisiones.
- Prepara datos confiables para predicción, segmentación o evaluación.

Esto es especialmente importante en áreas como finanzas (análisis de ingresos), marketing (datos de clientes) y administración (indicadores de desempeño).



CARACTERÍSTICAS DEL PREPROCESAMIENTO CON PANDAS

Manejo de valores faltantes	Eliminación de duplicados	Transformación de variables	Filtrado y selección	Consistencia de datos
Identificación y tratamiento de datos incompletos.	Evita distorsiones en análisis estadísticos.	Conversión de datos a formatos numéricos o categóricos.	Trabajo con subconjuntos relevantes de datos.	Uniformidad en nombres, formatos y escalas.

EJEMPLO: LIMPIEZA DE DATOS DE VENTAS

- Una empresa comercial desea analizar el desempeño de ventas mensuales de sus productos con el objetivo de apoyar la toma de decisiones de sus áreas, como la planificación de promociones, la asignación de presupuesto y la identificación de productos con bajo o alto rendimiento.
- La información de ventas ha sido extraída de diferentes fuentes internas, por lo que la base de datos presenta problemas de calidad, tales como valores faltantes en el monto de ventas y registros duplicados de algunos productos. Estos errores impiden realizar análisis confiables y pueden llevar a conclusiones equivocadas.

Producto	Ventas	Mes
A	1,500	Enero
B	2,300	Enero
C	—	Enero
B	2,300	Enero
C	—	Enero

EJEMPLO: LIMPIEZA DE DATOS DE VENTAS

- Para solucionar este problema, se utilizará Python con la biblioteca pandas, con la finalidad de limpiar y preparar los datos para asegurar que cada producto tenga un valor de ventas válido y que no existan duplicados.
- El resultado será una base de datos estructurada, lista para ser utilizada en análisis estadísticos, visualizaciones o modelos predictivos.
- Utilizando pandas en Python, resolveremos los siguientes aspectos:

- + **Identificar los valores faltantes en la columna “ventas”.**
- + **Reemplazar los valores faltantes por uno representativo (promedio de ventas).**
- + **Eliminar registros duplicados.**
- + **Obtener una base de datos limpia y coherente para el análisis.**

+ DESARROLLO DEL CÓDIGO EN PYTHON

Paso 1. Importar la biblioteca y crear el conjunto de datos

```
import pandas as pd
datos = {
    'Producto': ['A', 'B', 'C', 'B', 'C'],
    'Ventas': [1500, 2300, None, 2300, None],
    'Mes': ['Enero', 'Enero', 'Enero', 'Enero', 'Enero']
}
df = pd.DataFrame(datos)
print(df)
```

Resultado en Python: se observan valores faltantes (NaN) en la columna “ventas” y un registro duplicado del producto B.

	Producto	Ventas	Mes
0	A	1500.0	Enero
1	B	2300.0	Enero
2	C	NaN	Enero
3	B	2300.0	Enero
4	C	NaN	Enero

Paso 2. Identificación de valores faltantes

```
df.isnull()
```

Resultado en Python: la herramienta identifica claramente los registros con valores faltantes y permite decidir cómo tratarlos.

	Producto	Ventas	Mes
0	False	False	False
1	False	False	False
2	False	True	False
3	False	False	False
4	False	True	False

Paso 3. Tratamiento de valores faltantes

En este caso, se reemplazan los valores faltantes por el promedio de ventas.

```
promedio_ventas = df['Ventas'].mean()  
df['Ventas'] = df['Ventas'].fillna(promedio_ventas)  
print(df)
```

Resultado en Python: se asegura que todos los registros tengan valores numéricos para evitar errores en análisis posteriores.

	Producto	Ventas	Mes
0	A	1500.0	Enero
1	B	2300.0	Enero
2	C	2033.3	Enero
3	B	2300.0	Enero
4	C	2033.3	Enero

Paso 4. Eliminación de registros duplicados

```
df = df.drop_duplicates()  
print(df)
```

Resultado en Python: la base de datos queda limpia, sin duplicados, lo que garantiza un análisis más preciso.

	Producto	Ventas	Mes
0	A	1500.0	Enero
1	B	2300.0	Enero
2	C	2033.3	Enero

Paso 5. Revisión final de los datos

```
df.info()
```

Resultado en Python: la base de datos está completa, estructurada y lista para análisis estadístico o minería de datos.

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 3 entries, 0 to 2  
Data columns (total 3 columns):  
Producto      3 non-null object  
Ventas        3 non-null float64  
Mes           3 non-null object
```

+

OPTIMIZACIÓN: AJUSTE DE
HIPERPARÁMETROS

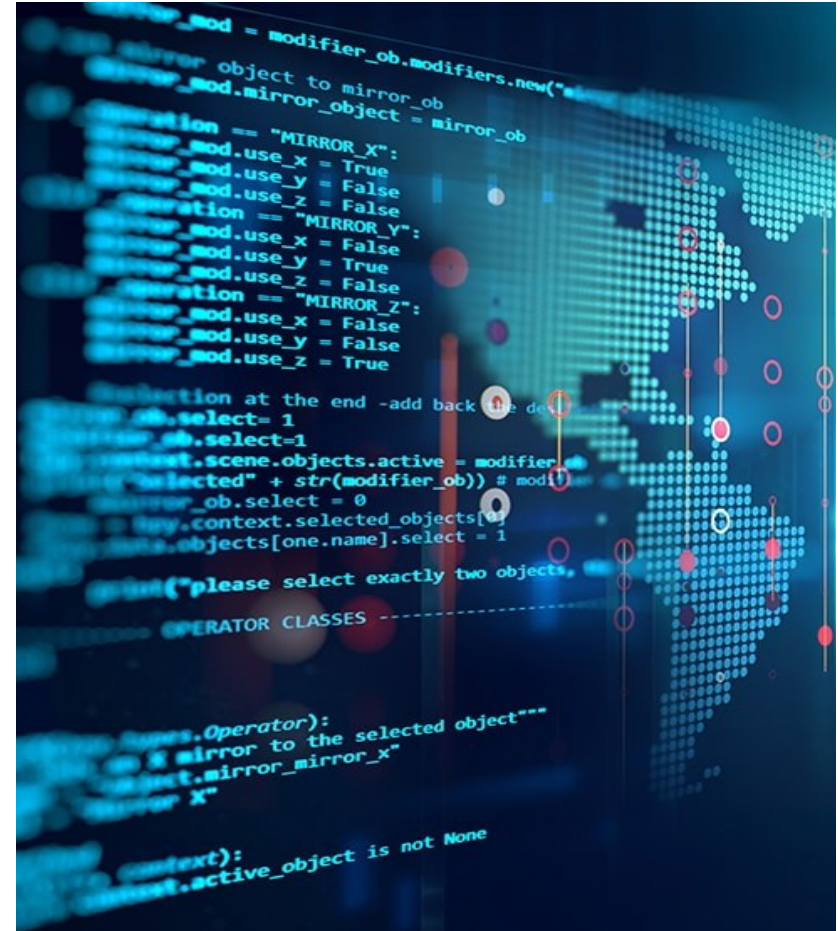
OPTIMIZACIÓN

- En minería de datos y aprendizaje automático, la optimización **consiste en mejorar el desempeño de un modelo predictivo para que genere resultados más precisos y confiables.**
- Esta mejora no se logra únicamente cambiando los datos, sino también ajustando ciertos parámetros que controlan el comportamiento del algoritmo.



HIPERPARÁMETROS

- Son **configuraciones externas al modelo que se definen antes del entrenamiento** y no se aprenden automáticamente a partir de los datos.
- Por ejemplo, la profundidad de un árbol de decisión o el número de vecinos en un modelo KNN. Cada combinación de hiperparámetros puede producir un modelo con resultados distintos.
- A diferencia de los parámetros internos (como los coeficientes de una regresión), **los hiperparámetros deben ser probados y evaluados de manera sistemática**. Un ajuste inadecuado puede generar modelos poco precisos o sobreajustados.



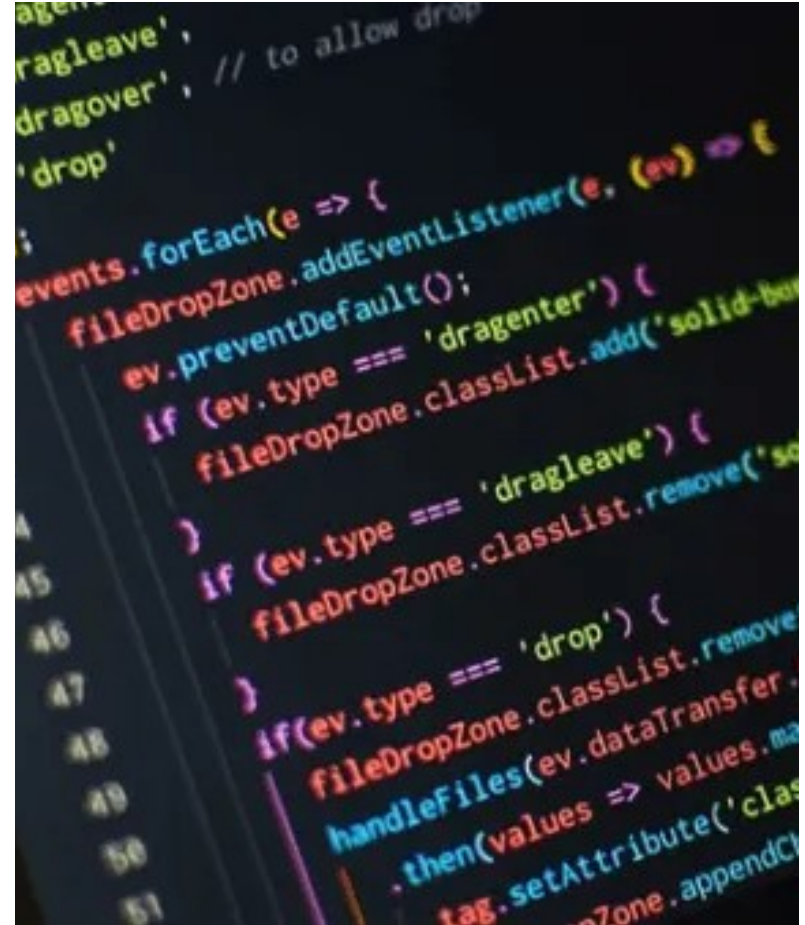
IMPORTANCIA EN EL ÁMBITO PROFESIONAL

- En contextos empresariales, un modelo mal optimizado puede llevar a decisiones incorrectas, como las siguientes:
 - **Clasificar erróneamente clientes.**
 - **Subestimar riesgos financieros.**
 - **Fallar en la predicción de ventas.**
- Por el contrario, un modelo bien optimizado incrementa la calidad de los análisis y la confianza en los resultados.
- En ese sentido, el ajuste de hiperparámetros es una competencia clave para los profesionales que trabajan con análisis de datos y modelos predictivos.



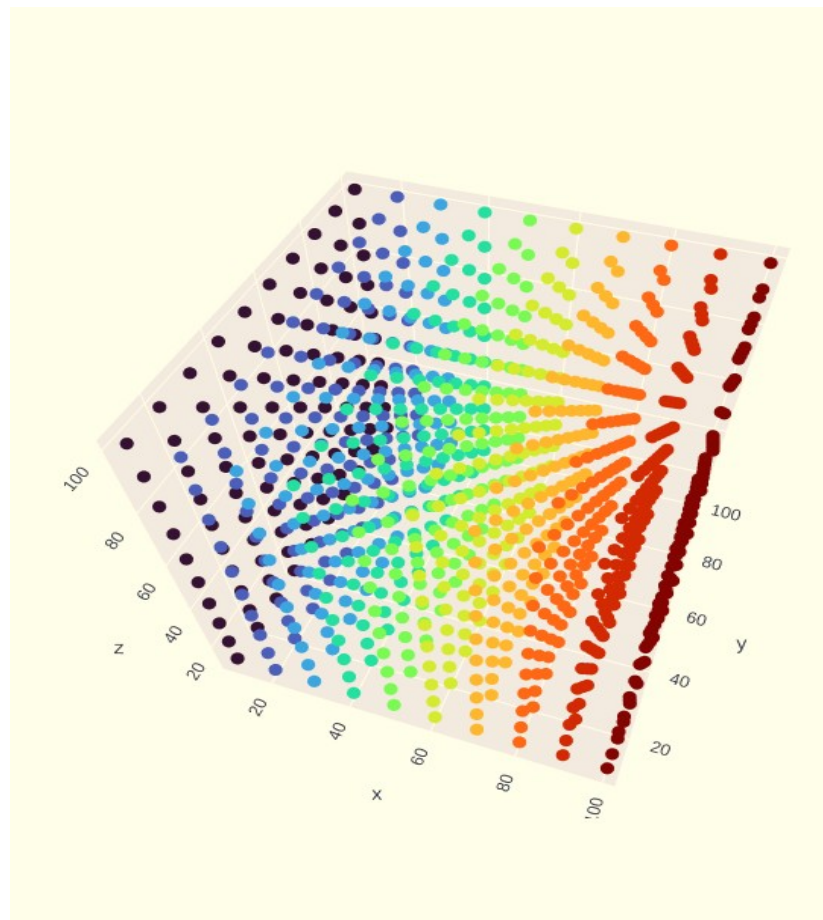
TÉCNICAS PRINCIPALES DE AJUSTE DE HIPERPARÁMETROS EN PYTHON

- En el desarrollo de modelos de minería de datos, no basta con elegir un algoritmo adecuado; es fundamental configurarlo correctamente.
- En Python, **la librería scikit-learn ofrece herramientas estandarizadas que permiten ajustar hiperparámetros** de forma sistemática, objetiva y reproducible.
- Las técnicas más utilizadas son las siguientes:
 - ▶ **GRID SEARCH**
 - ▶ **RANDOM SEARCH**
- Ambas están integradas con validación cruzada.



► GRID SEARCH (BÚSQUEDA EN MALLA)

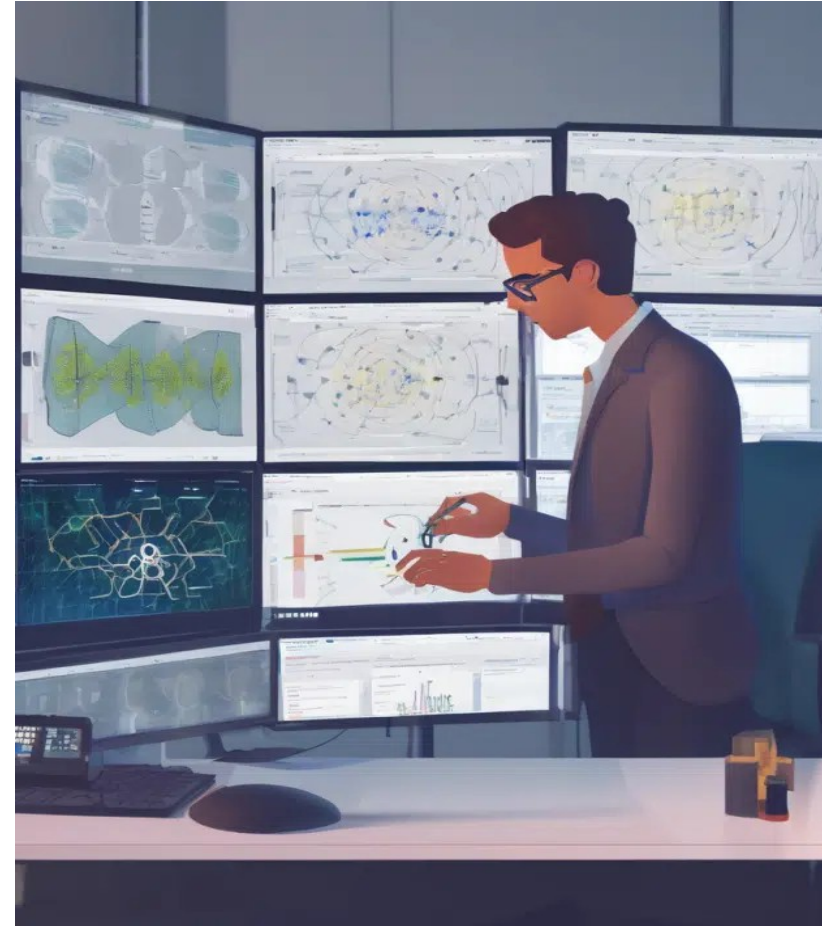
- Es una técnica exhaustiva que evalúa todas las combinaciones posibles de un conjunto previamente definido de hiperparámetros.
- El analista especifica qué valores desea probar para cada hiperparámetro y el algoritmo entrena y evalúa un modelo por cada combinación.
- Por ejemplo, en finanzas, Grid Search puede utilizarse para ajustar un modelo de clasificación de riesgo crediticio, en el que se desea probar un número limitado de configuraciones para asegurar la máxima precisión y confiabilidad del modelo antes de su implementación.



► GRID SEARCH (BÚSQUEDA EN MALLA)

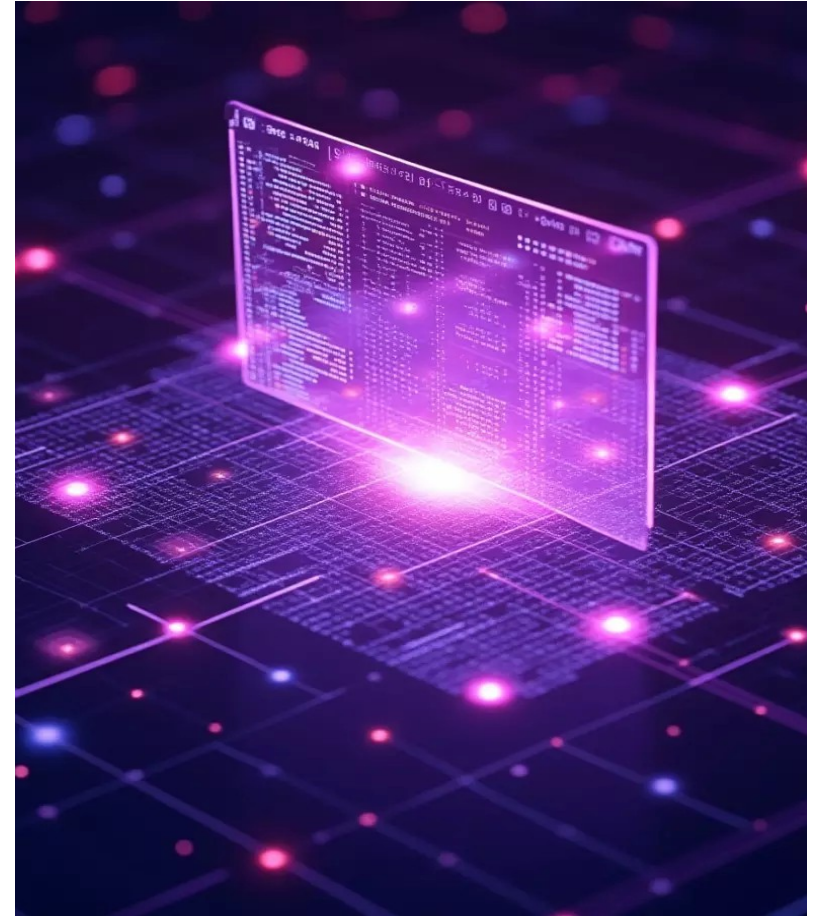
- **Características principales**

- + Es una búsqueda determinística y completa.
- + Garantiza encontrar la mejor combinación dentro del rango definido.
- + Puede ser computacionalmente costosa si se evalúan muchos hiperparámetros o rangos amplios.
- + Es ideal cuando el número de hiperparámetros es reducido o cuando se desea un control preciso del proceso.



▶ RANDOM SEARCH (BÚSQUEDA ALEATORIA)

- Evalúa un número definido de combinaciones aleatorias de hiperparámetros, extraídas de rangos o distribuciones especificadas. A diferencia de Grid Search, no prueba todas las combinaciones posibles.
- Por ejemplo, en marketing digital, Random Search es útil para optimizar modelos de predicción de clics o conversiones, en los que se manejan grandes volúmenes de datos y se requiere rapidez en la experimentación de modelos.



▶ RANDOM SEARCH (BÚSQUEDA ALEATORIA)

- **Características principales**

- + Es una búsqueda determinística y completa.
- + Reduce significativamente el tiempo de cómputo.
- + Permite explorar rangos amplios de hiperparámetros.
- + En muchos casos, logra resultados similares a Grid Search con menos evaluaciones.



COMPARACIÓN ENTRE GRID SEARCH VS. RANDOM SEARCH

CARACTERÍSTICA	GRID SEARCH	RANDOM SEARCH
Tipo de búsqueda.	Exhaustiva.	Aleatoria.
Tiempo de cómputo.	Alto.	Moderado o bajo.
Control del proceso.	Alto.	Medio.
Exploración del espacio.	Limitada a la malla.	Más amplia.
Recomendado cuando...	Pocos hiperparámetros.	Muchos hiperparámetros.

PROCESO DE DESARROLLO DEL AJUSTE DE HIPERPARÁMETROS EN PYTHON

- 01  Definir el problema y el algoritmo a utilizar.
- 02  Seleccionar los hiperparámetros que se desean optimizar.
- 03  Definir el rango de valores a evaluar.
- 04  Aplicar una técnica de búsqueda (Grid Search o Random Search).
- 05  Evaluar el modelo usando validación cruzada.
- 06  Seleccionar el mejor modelo y analizar los resultados.



EJEMPLO: PREDICCIÓN DE ABANDONO DE CLIENTES

- Una empresa de servicios desea predecir si un cliente abandonará el servicio (churn) a partir de variables como las siguientes:
 - **Antigüedad.**
 - **Consumo.**
 - **Número de reclamos.**
- Se utiliza un modelo de árbol de decisión y se optimizan sus hiperparámetros para mejorar la precisión del modelo.



+ PROGRAMACIÓN EN PYTHON

```
# Importación de librerías
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Datos simulados
data = {
    'antiguedad': [1, 3, 5, 2, 7, 8, 4, 6],
    'consumo': [200, 150, 300, 180, 400, 420, 210, 350],
    'reclamos': [3, 1, 0, 2, 0, 0, 1, 0],
    'abandono': [1, 0, 0, 1, 0, 0, 0, 0]
}

df = pd.DataFrame(data)

# Variables predictoras y variable objetivo
X = df[['antiguedad', 'consumo', 'reclamos']]
y = df['abandono']
```

+ PROGRAMACIÓN EN PYTHON

```
# División de datos
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Modelo base
modelo = DecisionTreeClassifier(random_state=42)
# Definición de hiperparámetros
parametros = {
    'max_depth': [2, 3, 4],
    'min_samples_split': [2, 4, 6]
}
# Grid Search con validación cruzada
grid = GridSearchCV(
    estimator=modelo,
    param_grid=parametros,
    cv=3,
    scoring='accuracy'
)
```

+ PROGRAMACIÓN EN PYTHON

```
# Entrenamiento del modelo optimizado
grid.fit(X_train, y_train)

# Mejor modelo
mejor_modelo = grid.best_estimator_

# Predicciones
y_pred = mejor_modelo.predict(X_test)

# Resultados
print("Mejores hiperparámetros:", grid.best_params_)
print("Exactitud:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

+ PROGRAMACIÓN EN PYTHON

Resultado - Salida aproximada

Mejores hiperparámetros: {'max_depth': 3, 'min_samples_split': 2}

Exactitud: 0.83

	<i>precision</i>	<i>recall</i>	<i>f1-score</i>	<i>support</i>
0	0.80	1.00	0.89	4
1	1.00	0.50	0.67	2
<i>accuracy</i>			0.83	6

+ **CONCLUSIONES**



x



x





x



x



+

BIBLIOGRAFÍA **MÁS REFERENCIAS**

- + Bruce, P., Bruce, A., & Gedeck, P. (2022). *Estadística práctica para ciencia de datos con R y Python* (2ª ed.). Marcombo / Alpha Editorial.
- + Ortega Candel, J. M. (2023). *Big data, machine learning y data science en Python*. Ediciones de la U.
- + Padilla Beltrán, J. E., & Contreras Bravo, L. E. (2024). *Ciencia de datos con Python: transformación y selección de variables*. Ediciones de la U.



ISIL+